CS Blood Donation Drive Coordinator

System Documentation

■# CS Blood Donation Drive Coordinator

1. Overview

The CS Blood Donation Drive Coordinator is a full-stack platform designed to digitize campus blood donation drives by replacing manual paper-based sign-up sheets with a real-time scheduling and operational dashboard. The system supports donor self-service booking, eligibility pre-screening, organizer scheduling, medical staff check-in and queue management, and system administration.

2. Problem Statement

Campus blood drives frequently experience double-booking, inefficient donor management, poor queue visibility, and unreliable donor tracking. Existing manual methods create issues such as conflicting appointments, lost records, no-shows, and limited analytics. The platform addresses these problems through a transactional database-backed scheduling engine and online workflows for all stakeholders.

3. Stakeholders and Roles

Donor

- Browse available blood drives
- Complete a pre-screening questionnaire
- Reserve exact time slots
- Track donation history and next eligibility date
- View active bookings and reminders

Drive Organizer / Coordinator

- Create and manage blood drives
- Configure time windows, capacities, and target donation units
- Track turnout and generate reports
- Broadcast alerts for critical donor needs

Medical Staff / Phlebotomist

- Check in donors onsite
- Manage queue status and walk-ins
- Record completed donations or deferrals
- Update donor outcomes in a live operational console

System Administrator

- Manage user accounts and roles
- Review audit logs
- Control access and platform configuration

4. System Goals

- Eliminate double-booking and scheduling errors.
- Streamline donor registration and pre-screening.
- Improve operational visibility for organizers and medical staff.
- Maintain auditable records for donation completion and deferral decisions.
- Support compliance-friendly reporting and export.

5. Architecture Overview

The system follows a three-layer architecture:

- Frontend: Next.js (App Router), React 19, TypeScript, Tailwind CSS.
- Backend: Node.js and Express.js REST API.
- Database: PostgreSQL using a connection pool and transactional locking.

Data flows from the client to the backend API, which validates requests and performs secure database transactions. Authentication is handled using JWTs and bcrypt-based password hashing.

6. Core Functional Modules

Donor Portal

The donor experience includes public drive browsing, pre-screening, and appointment booking. Users can check age, weight, travel history, and donation cooldown status before receiving an appointment slot.

Scheduling Engine

The drive scheduling system generates available time slots from the drive window and capacity configuration. Booking is protected by PostgreSQL transactional locks to prevent concurrent overbooking.

Check-in Console

Medical staff can search donors by name, email, or booking reference, verify arrival, and update status from booking to completion or deferral.

Reporting and Analytics

Organizers can view turnout metrics, donor counts, blood group distribution, and completion rates. Reviews and exports include CSV and PDF outputs.

Administration and Security

The platform uses role-based access control, secure environment configuration, JWT verification, and audit controls for sensitive operations. The project explicitly avoids exposing password hashes and sensitive medical answers to the browser.

7. Data Model Summary

The core tables include:

- users
- drives
- slots
- appointments
- donation_records
- notifications

Key rules include:

- Appointment records must reference a valid slot with remaining capacity.
- Capacity checks must happen within transactional database operations.
- Donation completion should update the donor's last donation date to support future eligibility calculations.

8. Security and Compliance Considerations

- Sensitive data is minimized and stored only when required.
- Backend-only logic handles booking decisions and role authorization.
- Password hashing occurs server-side using bcryptjs.
- JWTs are validated before protected endpoints are processed.
- CORS is configured explicitly and rate limiting is enabled for authentication routes.
- Medical pre-screen answers are not exposed beyond the donor's own required session.

9. Technology Stack

Frontend

- Next.js 16.3+
- React 19
- TypeScript
- Tailwind CSS v4
- Lucide React icons

Backend

- Node.js v20+
- Express.js 5.x
- JWT + bcryptjs
- dotenv
- Helmet and CORS
- Rate limiting

Database

- PostgreSQL
- pg connection pooling
- ACID transactions and row-level locking for concurrency-sensitive writes

10. Deployment and Operations

The project is designed for deployment across a frontend and backend split:

- Frontend: deployed to Vercel
- Backend: deployed to Render or similar Node hosting
- Database: PostgreSQL instance managed externally

Environment configuration includes database connection strings, JWT secret, frontend URL, and allowed origins.

11. Roadmap and Delivery Phases

The project plan is structured into sequential phases:

1. Architecture and authentication
2. Scheduling and booking
3. Eligibility and donor portal workflows
4. Day-of-drive check-in and queue management
5. Analytics, notifications, and security hardening
6. Deployment and production readiness

12. Conclusion

The CS Blood Donation Drive Coordinator is a production-oriented system that combines donor engagement, operational efficiency, and medical coordination into a unified digital platform. It is built to reduce friction during blood drives, improve donor turnout, and ensure data integrity and security in a

high-concurrency environment.

Source: Project README and Project Contract documents for the CS Blood Donation Drive Coordinator.